

Stack Overflow is a question and answer site for professional and enthusiast programmers. It's 100% free.

Take the 2-minute tour



How can I find the number of overlapping sequences in a String with Python?



I have a long sequence and I would like to know how often some subsequences occur in this sequence.

I know `string.count(s, sub)`, but it does only count non-overlapping sequences.

Does a similar function which counts also overlapping sequences exist?

python count

asked Jul 27 '11 at 12:07



moose

8,369 9 70 170

You might want to look at stackoverflow.com/questions/5616822/... – Matthew Schinckel Jul 27 '11 at 12:18

@moose Do you know the subsequences before to find their occurrence's counts? – eyquem Jul 27 '11 at 13:35

4 Answers

As an alternative to writing your own search function, you could use the `re` module:

```
In [22]: import re
In [23]: haystack = 'abababa baba alibababa'
In [24]: needle = 'baba'
In [25]: matches = re.finditer(r'(?=({s}))' % re.escape(needle), haystack)
In [26]: print [m.start(1) for m in matches]
[1, 3, 8, 16, 18]
```

The above prints out the starting positions of all (potentially overlapping) matches.

If all you need is the count, the following should do the trick:

```
In [27]: len(re.findall(r'(?=({s}))' % re.escape(needle), haystack))
Out[27]: 5
```

answered Jul 27 '11 at 13:03



NPE

210k 25 432 643

I'll vote for this one. It's much better than my own. A simple `len(matches)` will give you the result. – e-satis Jul 27 '11 at 13:06

one liner : `return len( re.finditer(r'(?=({s}))' % re.escape(needle), haystack)) ... superb :)` – DhruvPathak Jul 27 '11 at 13:21

@DhruvPathak `TypeError: object of type 'callable-iterator' has no len()` – eyquem Jul 27 '11 at 13:57



A simple to understand way to do it is:

```
def count(sub, string):
    count = 0
```

```

for i in xrange(len(string)):
    if string[i:].startswith(sub):
        count += 1
return count

count('baba', 'abababa baba alibababa')
#output: 5

```

If you like short snippets, you can make it less readable but smarter:

```

def count(subs, s):
    return sum((s[i:].startswith(subs) for i in xrange(len(s))))

```

This uses the fact that Python can treat boolean like integers.

edited Jul 27 '11 at 13:01

answered Jul 27 '11 at 12:54



e-satis
180k 73 211 267

With `Lsub = len(sub)` and `if string[i:i+Lsub]==sub:` and a string of length 298, it runs in 1.17 seconds while your code runs in 4 seconds (10000 turns of iteration) - Also, `sum()` is slow and the function with `sum()` is certainly slower (I didn't test though) - [eyquem](#) Jul 27 '11 at 14:11

My first version used `Lsub`, but I sacrificed speed for confort with this new version as the OP has never stated he had a perf requirement. I never profiled `sum` so I can't swear it's slow (I can't see a reason why it would be), but if it is, `itertools.reduce + operator.add` would do the job just fine. - [e-satis](#) Jul 27 '11 at 14:24

I've learnt the slowness of `sum()` from the member [gnibbler](#) in this thread: (stackoverflow.com/questions/3083829/...) (look in the comments). As you will see in these comments, it is the same reason that explains the slowness of `sum()` AND `reduce()` too - [eyquem](#) Jul 27 '11 at 14:40

That was my first assumption. But `itertools.reduce()` is not the same as `reduce()`: it doesn't have quadratic performance because it doesn't create a new list every time. All the functions in the `itertools` module use generators behind the scene. - [e-satis](#) Jul 28 '11 at 7:14

ahem... the problem is that there is no `reduce()` in the `itertools` module in the Python 2.7.1 doc, nor in Python 3.2 doc. In Python 3.2, `reduce()` is present in the `functools()` module only and it seems to have the same functioning as before in Python 2.x that is to say For example, `reduce(lambda x, y: x+y, [1, 2, 3, 4, 5])` calculates `((((1+2)+3)+4)+5)`. In Python 2.x, `functools.reduce()`: *This is the same function as `reduce()`. It is made available in this module to allow writing code more forward-compatible with Python 3.* - [eyquem](#) Jul 29 '11 at 9:27

This should help you :

```

matches = []
st = 'abababa baba alibababa'
needle = 'baba'
for i in xrange(len(st)-len(needle)+1):
    i = st.find(needle,i,i+len(needle))
    if(i >= 0):
        matches.append(st.find(needle,i,i+len(needle)))
print(str(matches))

```

see it here : <http://codepad.org/pmkKXmWB>

Did not benchmark it for long strings, see if its efficient enough for your use.

answered Jul 27 '11 at 12:21



DhruvPathak
17.5k 2 41 86

Would there be an advantage to replacing the lines of the loop with `... new_st = st[i:]; if new_st.startswith(needle): matches.append(i)` - [jcfollower](#) Jul 27 '11 at 12:46

Nope I guess, since `i` is already being used to check only particular chunk of the string `st.find(needle,i,i+len(needle))` starting from `i`, so creating substring and assigning would be just an overhead . - [DhruvPathak](#) Jul 27 '11 at 12:48

I learnt today that you could use a running index to fetch the next occurrence of your substring:

```

string = 'bobobobobobobob' # Long string or variable here
count = 0
start = 0
while True:
    index = string.find('bob', start)
    if index >= 0:
        count += 1
        start += 1
    else:
        break
print(count)

```

Returns 7

answered Sep 8 '14 at 10:30

[iamryandrake](#)124 2 16
